

Formal Analysis of the Google Play Integrity Attestation in ProVerif

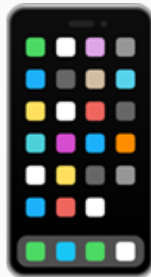
JosephALLEMAND, Étienne NEDJAÏ, Mohamed SABT

3 June 2026



Introduction

- ▶ Mobile apps are used for **sensitive operations**
e.g. banking, payment, identity verification, . . .
- ▶ The **environment cannot be trusted**:
 - ▶ Phones may be **rooted**
i.e. user processes gain root-level privileges
usually by **unlocking the bootloader**, and reinstalling OS
 - ▶ Apps may be **tampered** with
- ▶ **App providers** want guarantees on the execution environment



Android 

Integrity Guarantees

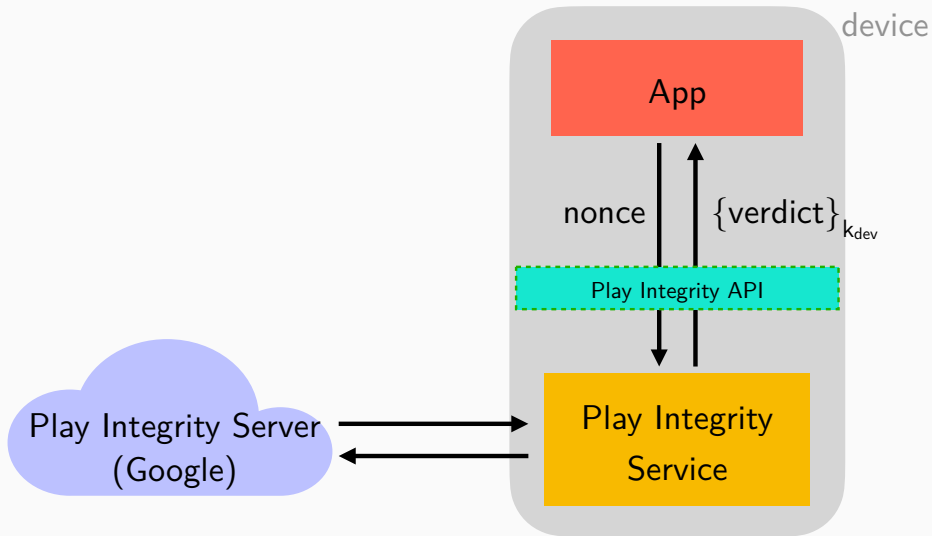
App providers can typically be interested in three kinds of guarantees:

- ▶ **Device Integrity:** ensure the device is **not rooted**
e.g. a banking app – bank wants to know other apps can't access its data
- ▶ **App Integrity:** ensure the app has **not been tampered** with
e.g. a mobile game – server wants to check the client is legitimate and not cheating
- ▶ **App Licensing:**
ensure app is **installed from PlayStore**, and the user's Google account **owns the app**
e.g. a paid app wants to check the user purchased it,
e.g. a geographically restricted app checks the user's account is in the correct area

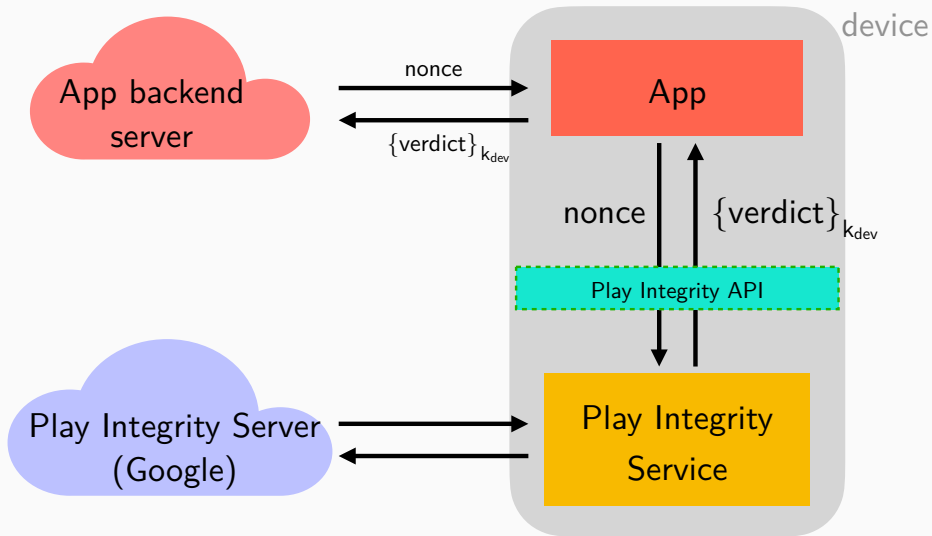
Google Play Integrity

- ▶ Apps **cannot** check their environment **by themselves**
e.g. a root-level attacker could provide fake information
- ▶ **Play Integrity** is a **service** provided by Google to Android apps:
An API devs can call to obtain **integrity verdicts** / **attestations**
- ▶ Uses **TEE** & communication with **Google servers**
- ▶ Only the **API is documented**, the underlying **protocol is proprietary**
in particular two things are **undocumented**:
 - ▶ **Structure and messages**: What data is sent to Google?
How does Android provide it to Play Integrity? How is it protected?
 - ▶ **Server behaviour**: How are verdicts determined?

Structure of Play Integrity

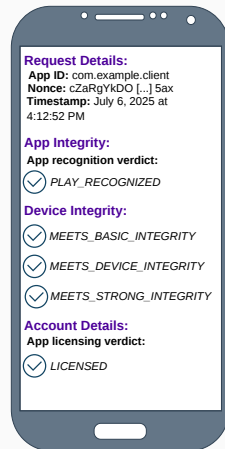


Structure of Play Integrity



Play Integrity verdicts

Field name	Possible values
App Integrity	PLAY_RECOGNIZED, UNRECOGNIZED_VERSION, UNEVALUATED
Device Integrity	MEETS_BASIC_INTEGRITY, MEETS_DEVICE_INTEGRITY, MEETS_STRONG_INTEGRITY, MEETS_VIRTUAL_INTEGRITY
Device Activity	LEVEL_1-LEVEL_4, UNEVALUATED
Account Details	LICENSED, UNLICENSED, UNEVALUATED
Env Details	KNOWN/UNKNOWN {INSTALLED, CAPTURING, CONTROLLING, OVERLAYS }
Env Details	NO_ISSUES, NO_DATA, POSSIBLE/MEDIUM/HIGH_RISK, UNEVALUATED

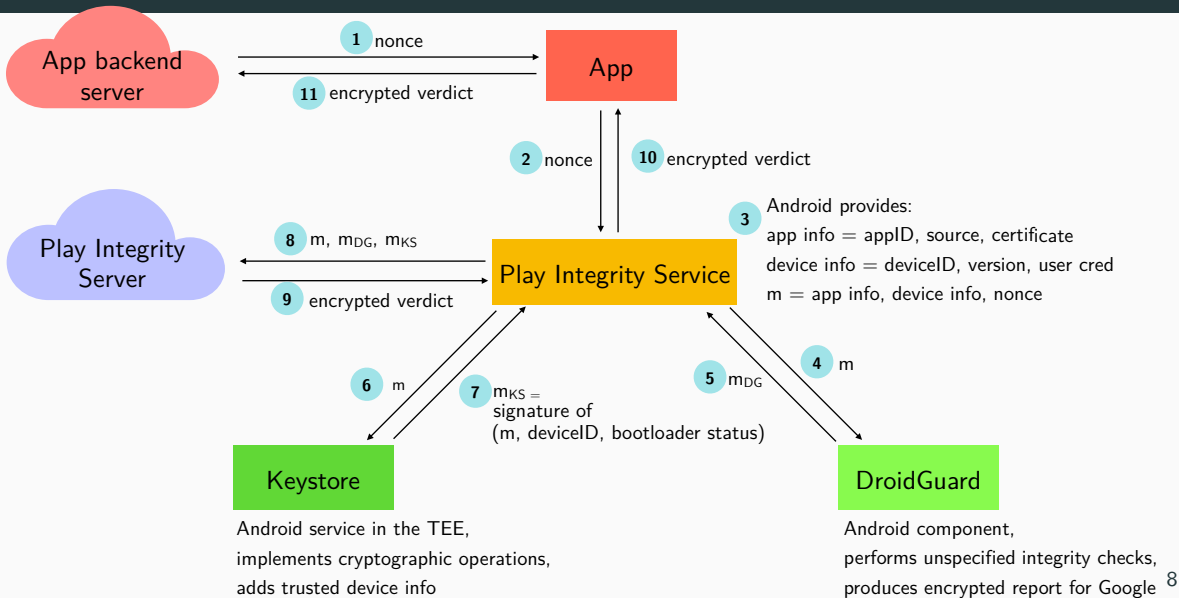


- ▶ Very granular verdicts
- ▶ Documentation provides informal descriptions, but no clear security specification

Our contributions

- ▶ **Reverse-engineer** the protocol:
what **messages** are sent, between which **services**,
what happens when **badly constructed** or unexpected messages are received,
how **verdicts** are chosen
- ▶ **Formalise** the reverse-engineered spec in **ProVerif** 
- ▶ Define clear **security goals** and **analyse** them with ProVerif
- ▶ Identify **two attacks**, **implement** them, propose and verify **mitigations**.

Reverse-engineered protocol: Structure



Reverse-engineered protocol: Server behaviour

For clearer presentation, we **condense** all server verdicts into **four booleans**

- ▶ v_1 : **unrooted device**, *i.e.* bootloader status is “locked”
- ▶ v_2 : **up-to-date device**, *i.e.* version info is recent
- ▶ v_3 : **uncompromised app**, *i.e.* app certificate provided is correct
- ▶ v_4 : **verified installation path**, *i.e.* app source = Play Store, user cred owns the app

Two crucial observations 🤖

- ▶ **DroidGuard's** report is actually **not used** by the server 🤔
Removing it, modifying it, setting it to 0 changes nothing to the response
- ▶ The server **does not check** that Keystore and Play Integrity ID is **consistent**

Formal Analysis in ProVerif

- ▶ We model the protocol in ✨ProVerif✨
(~ 1500 lines total, ~ 2min verification time)
- ▶ Processes for **each component** modelling behaviours inferred from our observations
- ▶ Arranged together to model the **scenario** we want to consider
- ▶ We consider the point of view of the **app provider** who wants guarantees regarding the **environment** its app runs in
- ▶ The **attacker** owns **any number of devices**, **rooted or not**, and can install **any number of apps** of their choice



Modelling details

- ▶ **Modular model**: all devices (**rooted** or **not**) use the same process, with **public** or **private** channels
- ▶ Several checks by Google's server rely on its **internal databases**
e.g. checking user is authorised, checking app certificate
We **overapproximate** them to avoid modelling the database:
just **ask the attacker** for the result + restriction to ensure **consistency**
 - ▶ **Simpler** to model and to update
 - ▶ Makes **less assumptions** on the server's behaviour, which we can't really know
 - ▶ **Faster** verification time

Analysis results: basic security goals

We formulate 6 security goals to characterise the expected guarantees

The first three are simple

Goal 1: Verdict Integrity

App receives and decrypts verdict $\Rightarrow$ verdict was generated by Google

Goal 2: Freshness

Encrypted verdicts cannot be reused (no replay)

Goal 3: App Binding

App receives and decrypts verdict $\Rightarrow$ verdict generated by Google **for this app**

Analysis results: basic security goals

We formulate 6 security goals to characterise the expected guarantees

The first three are simple , and we prove they hold (no surprises)

Goal 1: Verdict Integrity ✓

App receives and decrypts verdict $\Rightarrow$ verdict was generated by Google

Goal 2: Freshness ✓

Encrypted verdicts cannot be reused (no replay)

Goal 3: App Binding ✓

App receives and decrypts verdict $\Rightarrow$ verdict generated by Google **for this app**

Analysis results: Ideal case

Other goals describe what is obtained when **verdicts are checked**.

Goal 4: Ideal scenario

$$v_1 \wedge v_2 \wedge v_3 \wedge v_4 \Rightarrow \text{everything is fine}$$

i.e. device not rooted (v_1), up-to-date (v_2), app certificate checks out (v_3), installed from Play Store for authorised account (v_4)

This is the **strongest level** of integrity the app dev can ask for.

Analysis results: Ideal case

Other goals describe what is obtained when **verdicts are checked**.

Goal 4: Ideal scenario

$v_1 \wedge v_2 \wedge v_3 \wedge v_4 \Rightarrow$ everything is fine

i.e. device not rooted (v_1), up-to-date (v_2), app certificate checks out (v_3),
installed from Play Store for authorised account (v_4)

This is the **strongest level** of integrity the app dev can ask for.

Good news: it works! 🥳

Analysis results: Ideal case

Other goals describe what is obtained when **verdicts are checked**.

Goal 4: Ideal scenario

$v_1 \wedge v_2 \wedge v_3 \wedge v_4 \Rightarrow$ everything is fine

i.e. device not rooted (v_1), up-to-date (v_2), app certificate checks out (v_3),
installed from Play Store for authorised account (v_4)

This is the **strongest level** of integrity the app dev can ask for.

Good news: it works! 🥳

We could stop there, but this completely misses the **granularity** of Play Integrity:
in real life, devs often **do not want to check all verdicts**

Analysis results: more realistic expectations

Goal 5: only checking v_1

$v_1 \Rightarrow$ device is not rooted

e.g. a developer only cares that the device is not rooted,
but is fine with the app not being installed from Play Store

Analysis results: more realistic expectations

Goal 5: only checking v_1

$v_1 \Rightarrow$ device is not rooted

e.g. a developer only cares that the device is not rooted,
but is fine with the app not being installed from Play Store

Goal 6: everything except v_2

$v_1 \wedge v_3 \wedge v_4 \Rightarrow$ device not rooted, app certificate and installation source ok

e.g. a developer wants to allow outdated devices, but checks everything else they can

Analysis results: more realistic expectations

Goal 5: only checking v_1 ❌

$v_1 \Rightarrow$ device is not rooted

e.g. a developer only cares that the device is not rooted,
but is fine with the app not being installed from Play Store

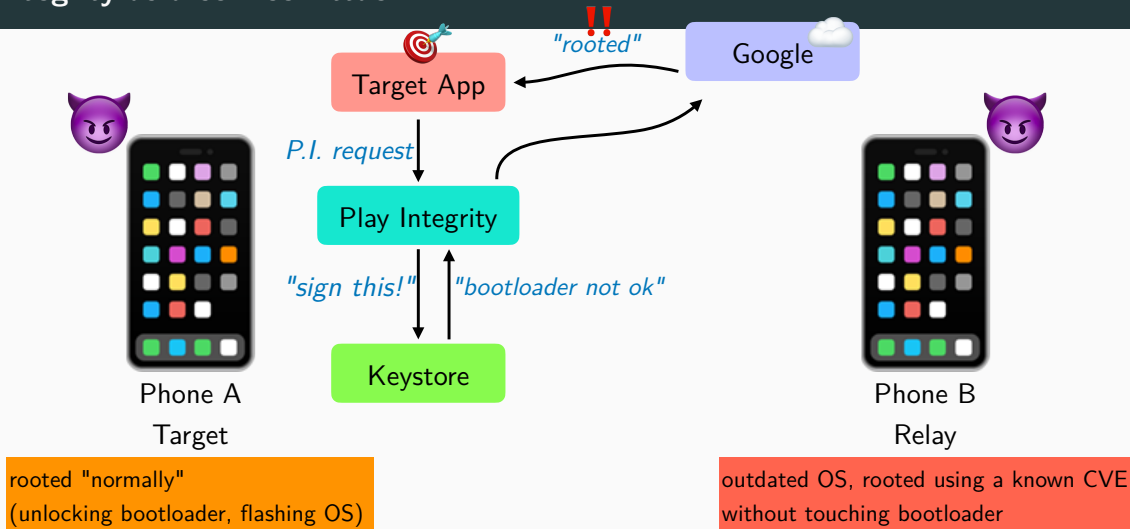
Goal 6: everything except v_2 ❌

$v_1 \wedge v_3 \wedge v_4 \Rightarrow$ device not rooted, app certificate and installation source ok

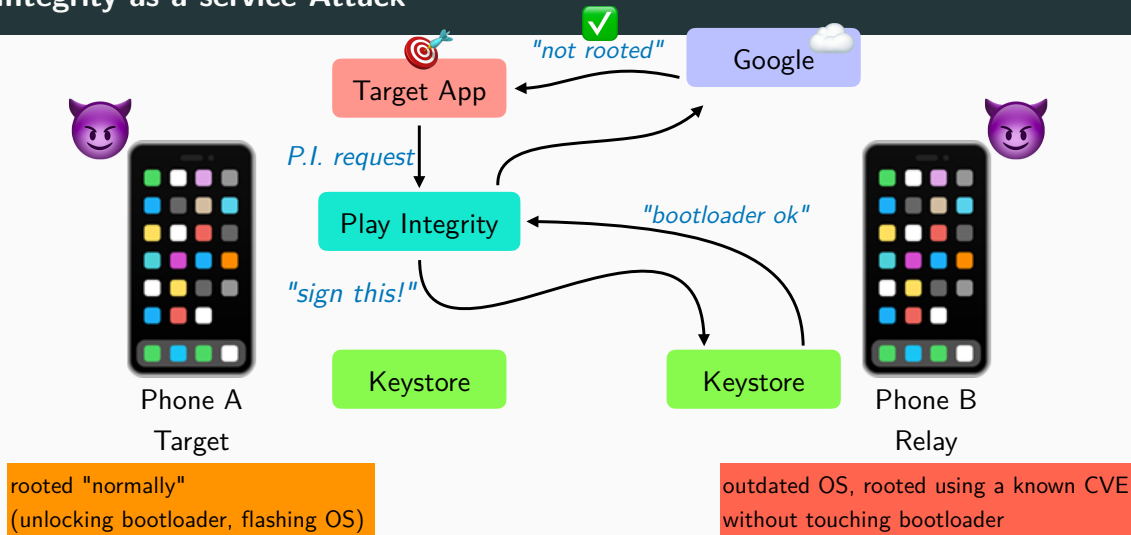
e.g. a developer wants to allow outdated devices, but checks everything else they can

- ▶ Bad news: it doesn't work! 🙄
- ▶ The granularity actually doesn't work
- ▶ Attacks, partial mitigations, updated model, verification

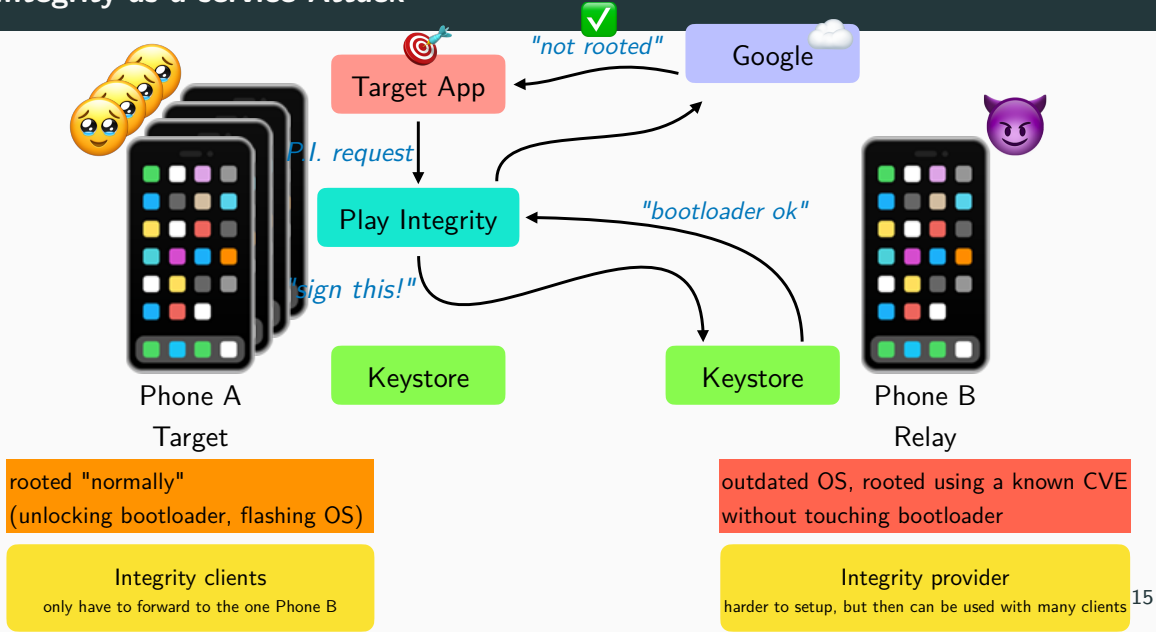
Integrity-as-a-service Attack



Integrity-as-a-service Attack



Integrity-as-a-service Attack



Conclusions: formal methods and reverse engineering

- ▶ Not a linear process: **back and forth** between the two
- ▶ A very productive **interaction** between the two aspects
 - ▶ Incorporating **newly observed behaviours** into the ProVerif model on the fly
 - ▶ Reverse engineering and practical understanding of the protocol **informs the way properties and model are written**
 - ▶ Formal modelling helps identify **unclear behaviours** that need to be further examined
- ▶ Only possible thanks to ProVerif's **automation and efficiency**

Questions?

